Wiki PWA

Technical Documentation & Architecture Guide

A Progressive Web Application for Knowledge Management

Version 1.0

Generated: April 21, 2026

Table of Contents

1. Executive Summary

Wiki PWA is a Progressive Web Application designed for offline-capable knowledge management. Built with Next.js 15, React, and TypeScript, it provides a modern, responsive interface for browsing, searching, and reading educational content.

The application features full PWA support including offline functionality, installability on any device, and automatic caching of content. It uses a modular architecture that allows easy customization of the dataset through a simple JSON file.

Key Features

- Offline-first architecture with service worker caching
- Installable as a native app on desktop and mobile
- Full-text search across all articles
- Responsive design for all screen sizes
- Custom dataset support via JSON configuration
- Markdown-like content rendering

2. System Architecture Overview

The Wiki PWA follows a modern JAMstack architecture pattern, combining static generation with dynamic client-side functionality. The system is composed of four main layers:

Architecture Layers

- Presentation Layer: React components with Tailwind CSS styling
- Application Layer: Next.js App Router with server and client components
- Data Layer: JSON-based data storage with TypeScript interfaces
- PWA Layer: Service worker and manifest for offline capability

Technology Stack

- Framework: Next.js 15 (App Router)
- Language: TypeScript
- Styling: Tailwind CSS v4
- UI Components: shadcn/ui
- Icons: Lucide React
- PWA: Custom service worker + Web App Manifest

3. Frontend Architecture

Component Hierarchy

The frontend follows a hierarchical component structure with clear separation of concerns:

```
app/
% % %  page.tsx          (Landing Page)
% % %  wiki/
%    % % %  page.tsx      (Wiki Application)
% % %  docs/
%    % % %  page.tsx      (User Documentation)
%    % % %  developer/
%       % % %  page.tsx  (Developer Documentation)
% % %  api/
    % % %  articles/     (REST API Routes)
```

State Management

The application uses React's built-in state management with useState and useMemo hooks. The main wiki page maintains three key pieces of state:

- searchQuery: Current search filter text
- selectedId: Currently selected article ID
- sidebarOpen: Mobile sidebar visibility state

Rendering Strategy

Components are marked with 'use client' directive when they require browser APIs or user interaction. The landing page and documentation pages leverage server components for better performance.

4. Backend Architecture

API Routes

The backend consists of Next.js API routes that provide RESTful endpoints:

```
GET /api/articles
  - Returns all articles
  - Query params: ?q=search_term
  - Optional: ?meta=true for full metadata

GET /api/articles/[id]
  - Returns single article by ID
  - Returns 404 if not found
```

Data Loader Module

The data-loader.ts module provides a clean abstraction over the JSON data file, offering functions for:

- getAllArticles(): Retrieve all articles
- getArticleById(id): Fetch specific article
- searchArticles(query): Full-text search
- getCategories(): List all categories
- getWikiMeta(): Get wiki metadata

5. PWA Implementation

Service Worker (sw.js)

The service worker implements a network-first caching strategy with fallback to cache, ensuring fresh content when online and cached content when offline.

```
Caching Strategy:
1. Try network request first
2. On success: Cache response, return to user
3. On failure: Return cached version
4. For navigation: Fall back to /wiki page
```

Web App Manifest

The manifest.json file defines the PWA properties:

- App name and description
- Theme and background colors
- Display mode (standalone)
- App icons (192x192, 512x512)
- Start URL (/wiki)

Install Prompt Hook (use-pwa.ts)

The usePWA hook manages the entire PWA lifecycle including service worker registration, offline detection, and the install prompt. It captures the beforeinstallprompt event to enable a custom install button.

6. Data Flow & State Management

Data Flow Diagram

The application follows a unidirectional data flow pattern:

```
JSON File (wiki-data.json)
       !“
Data Loader (data-loader.ts)
       !“
API Routes (/api/articles)
       !“
React Components
       !“
User Interface
```

Search Flow

- User types in SearchBar component
- searchQuery state updates in parent WikiPage
- useMemo recalculates filteredArticles
- ArticleList re-renders with filtered results

Article Selection Flow

- User clicks article in ArticleList
- onSelect callback fires with article ID
- selectedId state updates in WikiPage
- ArticleView receives and displays selected article
- Mobile: Sidebar closes automatically

7. Component Breakdown

SearchBar Component

A controlled input component that filters articles in real-time.

```
Props:
- value: string (current search query)
- onChange: (value: string) => void
```

ArticleList Component

Displays a scrollable list of article cards with metadata.

```
Props:
- articles: Article[] (filtered list)
- selectedId: string | null
- onSelect: (id: string) => void
```

ArticleView Component

Renders article content with markdown-like formatting support.

```
Props:
- article: Article | null
- onClose: () => void

Features:
- Heading detection (## and ###)
- Code block rendering
- List item formatting
- Tag display
- Reading time estimate
```

InstallButton Component

PWA install prompt button that adapts to installation state.

```
Props:
- canInstall: boolean
- isInstalled: boolean
- onInstall: () => Promise<void>
```

8. API Reference

Article Type Interface

```
interface Article {
  id: string;
  title: string;
  content: string;
  tags: string[];
  category: string;
  author: string;
  createdAt: string;
  difficulty?: "beginner" | "intermediate" | "advanced";
}
```

Category Type Interface

```
interface Category {
  id: string;
  name: string;
  description: string;
  icon: string;
}
```

WikiData Type Interface

```
interface WikiData {
  meta: {
    title: string;
    description: string;
    version: string;
    lastUpdated: string;
  };
  categories: Category[];
  articles: Article[];
}
```

9. File Structure

```
/vercel/share/v0-project/
% % %  app/
%    % % %  layout.tsx           # Root layout with PWA meta
%    % % %  page.tsx             # Landing/promotional page
%    % % %  globals.css          # Global styles
%    % % %  wiki/
%    %    % % %  page.tsx         # Main wiki application
%    % % %  docs/
%    %    % % %  page.tsx         # User documentation
%    %    % % %  developer/
%    %       % % %  page.tsx     # Developer documentation
%    % % %  api/
%       % % %  articles/
%           % % %  route.ts     # GET all articles
%           % % %  [id]/
%               % % %  route.ts # GET single article
% % %  components/
%    % % %  ui/                  # shadcn/ui components
%    % % %  wiki/
%       % % %  search-bar.tsx
%       % % %  article-list.tsx
%       % % %  article-view.tsx
%       % % %  install-button.tsx
% % %  hooks/
%    % % %  use-pwa.ts          # PWA functionality hook
%    % % %  use-articles.ts     # SWR data fetching hook
% % %  lib/
%    % % %  types.ts            # TypeScript interfaces
%    % % %  data-loader.ts      # Data access layer
%    % % %  utils.ts            # Utility functions
% % %  public/
    % % %  manifest.json       # PWA manifest
    % % %  sw.js               # Service worker
    % % %  icon-192x192.png    # PWA icon
    % % %  icon-512x512.png    # PWA icon
    % % %  data/
        % % %  wiki-data.json  # Article database
```

10. Customization Guide

Adding New Articles

To add new articles, edit the public/data/wiki-data.json file:

```
{
  "id": "unique-article-id",
  "title": "Article Title",
  "content": "Article content with ## headings...",
  "tags": ["tag1", "tag2"],
  "category": "category-id",
  "author": "Author Name",
  "createdAt": "2024-01-15",
  "difficulty": "beginner"
}
```

Adding New Categories

- Add category object to categories array in JSON
- Use consistent ID format (lowercase, hyphens)
- Reference category ID in article's category field

Customizing Appearance

- Edit globals.css for theme colors and fonts
- Modify Tailwind classes in components
- Update manifest.json for PWA branding

Deployment

The application can be deployed to any platform supporting Next.js:

- Vercel (recommended): One-click deployment
- Netlify: With Next.js adapter
- Self-hosted: Using Node.js server
